# Marsrover – Befehle

Kurzanleitung für die Library `rover.py` – Motoren, Sensoren und Servos

## 1. Start: immer oben in main.py

```
from rover import Rover
rover = Rover()
```

## 2. Fahren

Tempo immer **0 bis 100**. Mit `seconds=` fährt der Rover so lange und stoppt dann selbst. Ohne `seconds` fährt er weiter, bis ein neuer Befehl kommt oder main.py zu Ende ist.

| Befehl | Was passiert |
|---|---|
| **`rover.forward(50)`** | vorwärts, fährt weiter |
| **`rover.forward(50, seconds=2)`** | 2 Sekunden vorwärts, dann Stopp |
| **`rover.backward(50)`** | rückwärts |
| **`rover.backward(50, seconds=2)`** | 2 Sekunden rückwärts, dann Stopp |
| **`rover.turn_left(50)`** | an Ort nach links drehen |
| **`rover.turn_right(50)`** | an Ort nach rechts drehen |
| **`rover.turn_left(50, seconds=1)`** | 1 Sekunde links drehen, dann Stopp |
| **`rover.drive(60, 30)`** | links 60, rechts 30 -> Kurve nach rechts |
| **`rover.drive(-50, 50)`** | negative Zahl = diese Seite rückwärts |
| **`rover.drive(60, 30, seconds=2)`** | Kurve 2 Sekunden lang, dann Stopp |
| **`rover.stop()`** | alle Motoren aus |
| **`rover.wait(1.5)`** | 1.5 Sekunden warten – Motoren laufen weiter! |

## 3. Mit der Tastatur steuern

| Befehl | Was passiert |
|---|---|
| **`rover.wasd()`** | W / A / S / D fahren, Leertaste = Stopp, Q = beenden |
| **`rover.wasd(speed=40)`** | dasselbe, langsamer |

Geht nur, wenn du main.py selbst im Terminal startest (der Auto-Pull-Dienst hat keine Tastatur):

```
sudo systemctl stop marsrover
python3 ~/Marsrover/main.py
```

## 4. Sensoren und Ausgänge am GrovePi – Port selber wählen

Du entscheidest, was wo eingesteckt ist. Analoge Ports: **A0, A1, A2**. Digitale Ports: **D2 bis D8**. Jeder Port darf nur einmal benutzt werden.

| Befehl | Was passiert |
|---|---|
| `licht = rover.analog("A1")` | analoger Sensor (z.B. Licht) an A1 |
| `licht.read()` | Rohwert 0 bis 1023 |
| `abstand = rover.ultrasonic("D4")` | Ultraschall-Sensor an D4 |
| `abstand.read()` | Abstand in cm |
| `taster = rover.digital_in("D3")` | digitaler Eingang (z.B. Taster) an D3 |
| `taster.read()` | 0 oder 1 |
| `led = rover.digital_out("D5")` | digitaler Ausgang (z.B. LED) an D5 |
| `led.on() / led.off()` | einschalten / ausschalten |

Braucht die GrovePi-Library (Dexter Industries) auf dem Pi. Fahren geht auch ohne sie.

## 5. Servos (Grove 16-Kanal PWM-Treiber)

Kanal **0 bis 15** und I2C-Adresse wählst du selber. Adresse mit `i2cdetect -y 1` prüfen (Seeed-Standard: 0x7F).

| Befehl | Was passiert |
|---|---|
| `servo = rover.servo(channel=0, address=0x7F)` | Servo an Kanal 0 |
| `servo.angle(90)` | auf 90 Grad fahren (0 bis 180) |
| `servo.off()` | Servo stromlos – hält nicht mehr |
| `rover.servo(channel=1, address=0x7F, min_us=600, max_us=2400)` | grösserer Pulsbereich = mehr Weg |

Standard ist vorsichtig 1000–2000 µs (dreht evtl. weniger als 180 Grad). Bereich langsam erweitern. Brummt der Servo am Ende oder drückt gegen den Anschlag -> Bereich wieder kleiner machen!

## 6. Einstellungen

| Befehl | Was passiert |
|---|---|
| `Rover(addresses=[0x0F, 0x0E])` | zwei Motortreiber (Adressen mit `i2cdetect -y 1` prüfen) |
| `Rover(addresses=[])` | ohne Motortreiber – nur Sensoren/Servos testen |
| `Rover(max_speed=60)` | Höchsttempo in % der Akkuspannung. DG01D sind 4.5-V-Motoren: bei 7.4 V nicht über ca. 60 |
| `Rover(invert_right=True)` | rechte Seite dreht falsch herum (Rover dreht bei forward() an Ort) |
| `Rover(invert_left=True)` | linke Seite dreht falsch herum |
| `Rover(swap_sides=True)` | Motoren andersrum eingesteckt: J1 = rechts, J5 = links |
| `Rover(grovepi_address=0x04)` | I2C-Adresse des GrovePi (Standard 0x04) |

Mehrere Einstellungen gehen zusammen, z.B. `Rover(addresses=[0x0F], max_speed=50, swap_sides=True)`

## 7. Beispiel main.py

```
from rover import Rover

rover = Rover()
abstand = rover.ultrasonic("D4")
arm = rover.servo(channel=0, address=0x7F)

arm.angle(90)
rover.forward(40)                  # losfahren
while abstand.read() > 20:         # bis ein Hindernis naeher als 20 cm ist
    rover.wait(0.1)
rover.stop()
rover.turn_left(50, seconds=1)
```

## 8. Terminal-Befehle auf dem Pi

| Befehl | Was passiert |
|---|---|
| **`python3 ~/Marsrover/main.py`** | Code von Hand starten (Ctrl+C = abbrechen) |
| **`sudo systemctl stop marsrover`** | Auto-Pull + Auto-Start pausieren |
| **`sudo systemctl start marsrover`** | Auto-Pull wieder einschalten |
| **`journalctl -u marsrover -f`** | Ausgabe vom laufenden Code ansehen |
| **`i2cdetect -y 1`** | zeigt angeschlossene I2C-Geräte (Motortreiber = 0f) |

## 9. Gut zu wissen

• Verkabelung: Klemme **J1** (bei IN1/IN2) = linker Motor, Klemme **J5** (bei IN3/IN4) = rechter Motor (oder `swap_sides=True`).
• Motoren stoppen automatisch, wenn main.py endet, abstürzt oder abgebrochen wird. Servos und LEDs bleiben, wie sie sind.
• Nie Code direkt auf dem Pi ändern – der nächste Push überschreibt ihn.
• Beim Testen Rover hochstellen (Räder in der Luft)!
• Fehler „antwortet nicht“ -> Kabel prüfen, `i2cdetect -y 1`. Treiber Nr. 3 ist defekt.
• Fehler „Port ... ist schon belegt“ -> gleichen Port zweimal benutzt.
• Fehler „GrovePi-Library fehlt“ -> GrovePi-Software auf dem Pi installieren.